

APELLIDOS (en MAYÚSCULAS)

|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|

NOMBRE (en MAYÚSCULAS)

|  |  |  |  |  |  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|--|--|--|--|--|
|  |  |  |  |  |  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|--|--|--|--|--|

Cada hoja se entrega por separado y es obligatorio entregar todas las hojas. Se debe poner el **nombre completo en todas las hojas**. No se podrá entregar hasta pasados **20 minutos**. No se pueden usar hojas de sucio, para ello se dispone del espacio sobrante en cada hoja. El examen tendrá una duración máxima de **1:30 h**. No se podrá salir de los bucles usando **break o return**.

1.- (1 pto) Escribir un programa en C que lea caracteres de teclado ('#' para finalizar) e imprima por pantalla, al final, las dos últimas letras mayúsculas leídas según el orden de aparición. Si no aparece una o las dos, se imprimirá el carácter '\*' en su lugar. Ejemplos de secuencias de entrada y resultados a imprimir:

a\$%t5wrrmasGt67/()|\_!HhNmsR\*+mqX#

RX

a\$%t5wrrmasGt67#

G\*

#

\*\*

```
int main()
{ char penult = '*', ult = '*', act;

  printf ("Introduzca caracteres, # para acabar:\n");

  do
  { scanf("%c", &act);

    if (act>='A' && act <='Z')
      if (penult == '*')
        penult = act;
      else
        if (ult == '*')
          ult = act;
        else
          { penult = ult;
            ult = act;
          }
    } while (act != '#');

  printf ("\n\n%c %c\n", penult, ult);
}
```

2.- (1 pto) Implementar una función en C que, pasándole como parámetros una matriz de MxN caracteres (siendo M y N constantes ya definidas) y las coordenadas de dos posiciones dentro de la matriz (f1, c1, f2, c2) que definen correctamente una submatriz dentro de la matriz, ponga a '@' todas las posiciones de la matriz que queden fuera de la submatriz. Por ejemplo, para M=5, N=6, f1=1, c1=2, f2=3 y c2=4, partiendo de la matriz de la izda se deberá transformar como indica la matriz de la dcha:

|   |   |   |   |   |    |
|---|---|---|---|---|----|
| f | b | Ñ | 5 | < | =  |
| 4 | L | % | 3 | ¿ | "  |
| m | s | v | / | Ü | ö  |
| & | u | ç | ã | ß | 6  |
| ã | € | « | ® | À | \$ |

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| @ | @ | @ | @ | @ | @ |
| @ | @ | % | 3 | ¿ | @ |
| @ | @ | v | / | Ü | @ |
| @ | @ | ç | ã | ß | @ |
| @ | @ | @ | @ | @ | @ |

**NOTA:** No se pueden usar estructuras de datos auxiliares ni tampoco se podrán procesar las posiciones que pertenecen a la submatriz, es decir, los bucles que procesan la matriz no podrán pasar por las posiciones de la submatriz.

```
void cambiar_matr (char mat[M][N], unsigned f1, unsigned c1, unsigned f2, unsigned c2)
{ unsigned i, j;

  for (j=0; j<N; j++)
  { for (i=0; i<f1; i++)          // filas anteriores a la submatriz
    mat[i][j] = '@';
    for (i=f2+1; i<M; i++)       // filas posteriores a la submatriz
    mat[i][j] = '@';
  }

  for (i=f1; i<=f2; i++)
  { for (j=0; j<c1; j++)          // columnas a la izda de la submatriz
    mat[i][j] = '@';
    for (j=c2+1; j<N; j++)       // columnas a la dcha de la submatriz
    mat[i][j] = '@';
  }
}
```

3.- (1 pto) Escribir una función en C que sume enteros positivos muy grandes almacenados en arrays de dimensión N, siendo N una constante ya definida. En cada posición del array se almacena un único dígito (valor entre 0 y 9). Por ejemplo, el número 3184697325810036 se almacenará en un array de N=20 así:

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 3 | 1 | 8 | 4 | 6 | 9 | 7 | 3 | 2 | 5 | 8 | 1 | 0 | 0 | 3 | 6 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

La función recibirá como parámetros dos arrays que representan los números a sumar y un tercero donde se dejará el resultado de la suma, y devolverá como resultado el acarreo de la suma (1 o 0 en función de que al sumar las dos últimas posiciones el valor obtenido sea >= 10 o no). Por ejemplo, para N=10 y los dos siguientes arrays:

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 7 | 3 | 2 | 5 | 8 | 1 | 0 | 0 | 3 | 6 |
| 8 | 4 | 6 | 9 | 7 | 3 | 2 | 5 | 8 | 1 |

el valor devuelto por la función es 1 (acarreo de la última suma) y el tercer array quedará:

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 5 | 7 | 9 | 5 | 5 | 4 | 2 | 6 | 1 | 7 |
|---|---|---|---|---|---|---|---|---|---|

APELLIDOS (en MAYÚSCULAS)

|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|

NOMBRE (en MAYÚSCULAS)

|  |  |  |  |  |  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|--|--|--|--|--|
|  |  |  |  |  |  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|--|--|--|--|--|

```
unsigned suma_grandes (unsigned *vect1, unsigned *vect2, unsigned *result)
{ unsigned acarreo=0, i;

  for (i=N-1; i>=0; i--)
  { result[i] = (vect1[i] + vect2[i] + acarreo) % 10;
    acarreo = (vect1[i] + vect2[i] + acarreo) / 10;
  }

  return (acarreo);
}
```

**4.- (1 pto) Implementar un programa en C que haga uso de la función del ejercicio anterior, para lo que será necesario declarar e inicializar las variables adecuadas (no se pueden inicializar los arrays al declararlos), e imprimir los valores del tercer array después de llamar a la función. Se considera definida la constante N.**

```
main ()
{ unsigned array1[N], array2[N], array3[N], i, acarreo;

  printf ("Introduzca los valores del primer número: ");
  for (i=0; i<N; i++)
    do
      { scanf ("%u", &array1[i]);
        } while (array1[i]>9);

  printf ("\n\nIntroduzca los valores del segundo número: ");
  for (i=0; i<N; i++)
    do
      { scanf ("%u", &array2[i]);
        } while (array2[i]>9);

  acarreo = suma_grandes (array1, array2, array3);

  printf ("\n\n");
  for (i=0; i<N && !array3[i]; i++); // se saltan los ceros de la izda
  if (acarreo)
    printf ("%d", acarreo);
  for ( ; i<N; i++)
    printf ("%d", array3[i]); // se imprimen los valores significativos del array
}
```